

Manual de instalación

MAIA News Analyzer

1. Objetivo

Este documento describe cómo instalar y ejecutar el prototipo MAIA News Analyzer en una instancia Linux, utilizando Docker y Docker Compose.

La solución está compuesta por:

- Frontend React servido con Nginx.
- Backend FastAPI.
- Modelo de clasificación empaquetado como artefacto Python .whl.
- Servidor MLflow para seguimiento de experimentos.
- DVC con almacenamiento remoto en Amazon S3 para versionado de datos.

2. Repositorios

Repositorio de aplicación, API, tablero y despliegue:

```
https://github.com/Micro-Proyecto-Desarrollo-de-Soluciones/news_analyzer
```

Repositorio de procesamiento, entrenamiento, modelos y DVC:

```
https://github.com/Micro-Proyecto-Desarrollo-de-Soluciones/Clasificacion-de-la-orientacion-politica
```

3. Requisitos

Se recomienda una instancia Ubuntu en AWS EC2.

Software requerido:

- Git
- Docker
- Docker Compose v2
- Python 3.12 y entorno virtual únicamente para tareas de entrenamiento/DVC
- AWS CLI para acceder al almacenamiento S3 cuando se requiera reproducir los datos

Puertos utilizados por la aplicación:

Servicio	Puerto
Frontend	5173
Backend FastAPI	8000
MLflow	8050

En AWS EC2, el Security Group debe permitir acceso a los puertos necesarios desde las direcciones autorizadas.

4. Clonación de los repositorios

```
cd ~
git clone https://github.com/Micro-Proyecto-Desarrollo-de-Soluciones/news_analyzer.git
git clone https://github.com/Micro-Proyecto-Desarrollo-de-Soluciones/Clasificacion-de-la-orientacion-politica.git
```

5. Instalación de Docker

Verificar primero si Docker ya está instalado:

```
docker --version
docker compose version
```

Si no está instalado, en Ubuntu puede instalarse con:

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

Después de agregar el usuario al grupo docker, debe cerrarse y abrirse nuevamente la sesión SSH.

6. Estructura de despliegue

El repositorio news_analyzer contiene el archivo:

```
docker-compose.yml
```

Este archivo orquesta tres servicios:

```
frontend
backend
mlflow
```

El backend incluye el paquete instalable del modelo:

```
model-pkg/modelo_sesgo-0.1.0-py3-none-any.whl
```

El paquete contiene la lógica necesaria para realizar la inferencia con el mismo preprocesamiento utilizado durante el entrenamiento.

7. Construcción y ejecución

Entrar al repositorio:

```
cd ~/news_analyzer
```

Construir y levantar todos los servicios:

```
docker compose up -d --build
```

Verificar su estado:

```
docker compose ps
```

Los servicios esperados son:

```
maia-news-frontend  
maia-news-backend  
maia-mlflow
```

El backend debe aparecer en estado `healthy`.

8. Verificación de servicios

Desde la instancia EC2:

```
curl -s -o /dev/null -w "Backend: ${http_code}\n" http://localhost:8000/api/health  
curl -s -o /dev/null -w "Frontend: ${http_code}\n" http://localhost:5173/  
curl -s -o /dev/null -w "MLflow: ${http_code}\n" http://localhost:8050/
```

La respuesta esperada es:

```
Backend: 200  
Frontend: 200  
MLflow: 200
```

9. Acceso desde navegador

Obtener la IP pública de la instancia:

```
curl -s https://checkip.amazonaws.com
```

Suponiendo una IP pública <IP_PUBLICA>, los servicios quedan disponibles en:

```
Frontend:  
http://<IP_PUBLICA>:5173/  
  
Swagger / API:  
http://<IP_PUBLICA>:8000/docs  
  
MLflow:  
http://<IP_PUBLICA>:8050/
```

10. Prueba de inferencia por API

El endpoint principal de inferencia es:

```
POST /api/predict
```

Ejemplo:

```
curl -X POST http://localhost:8000/api/predict \  
-H "Content-Type: application/json" \  
-d '{  
  "titulo": "Economic policy debate",  
  "texto": "The government announced a new economic plan focused on public investment,  
  healthcare access, education funding and support for low income families."  
}'
```

La respuesta incluye:

- clase predicha;
- probabilidades para left, center y right;
- versión del modelo;
- advertencias;
- explicación de términos relevantes.

También puede probarse el flujo utilizado por el tablero:

```
POST /api/articles/analyze
```

Este endpoint acepta texto o URL de un artículo y delega el preprocesamiento y clasificación al modelo empaquetado.

11. Uso del tablero

Abrir:

```
http://<IP_PUBLICA>:5173/
```

El usuario puede ingresar un artículo o texto y solicitar el análisis.

El frontend envía la entrada al backend FastAPI. El backend utiliza el paquete del modelo para devolver la orientación estimada, probabilidades, nivel de confianza y elementos explicativos.

12. MLflow

MLflow se ejecuta como servicio independiente dentro de Docker Compose.

El servicio utiliza volúmenes persistentes:

```
mlflow-data  
mlflow-artifacts
```

Esto permite conservar la base de seguimiento y los artefactos aunque los contenedores sean reiniciados.

Para revisar los logs:

```
docker compose logs -f mlflow
```

Para levantar únicamente MLflow:

```
docker compose up -d mlflow
```

También existe el script:

```
./scripts/setup_mlflow.sh
```

que construye e inicia el servicio MLflow.

13. Reproducibilidad de datos con DVC

Los datos procesados se encuentran versionados mediante DVC en el repositorio:

```
Clasificacion-de-la-orientacion-politica
```

El almacenamiento remoto configurado utiliza Amazon S3:

```
s3://maia-17-dvcstore/dvc
```

Para trabajar con DVC se recomienda utilizar un entorno virtual:

```
python3 -m venv ~/.venv
source ~/.venv/bin/activate
pip install "dvc[s3]"
```

Las credenciales AWS deben proporcionarse mediante variables de entorno y nunca almacenarse en Git:

```
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_SESSION_TOKEN="..."
export AWS_DEFAULT_REGION="us-east-1"
```

Validar acceso:

```
aws sts get-caller-identity
```

Verificar el remote:

```
dvc remote list
```

Comprobar sincronización:

```
dvc status -c
```

Recuperar los datos versionados:

```
dvc pull
```

Los archivos versionados incluyen los conjuntos procesados empleados para entrenamiento, validación y prueba.

14. Actualización de la aplicación

Para desplegar una versión nueva:

```
cd ~/news_analyzer
git pull --ff-only origin main
docker compose up -d --build
```

Después verificar:

```
docker compose ps
```

15. Detener los servicios

Para detener temporalmente la solución sin eliminar los contenedores:

```
docker compose stop
```

Para volver a iniciarla:

```
docker compose start
```

Para retirar los contenedores sin eliminar los volúmenes persistentes:

```
docker compose down
```

No utilizar:

```
docker compose down -v
```

si se desea conservar la información persistente de MLflow.

16. Diagnóstico básico

Estado de servicios:

```
docker compose ps
```

Logs del backend:

```
docker compose logs backend
```

Logs del frontend:

```
docker compose logs frontend
```

Logs de MLflow:

```
docker compose logs mlflow
```

Validar API:

```
curl http://localhost:8000/api/health
```

Si un servicio no puede ser accedido desde Internet, verificar:

- que el contenedor esté en ejecución;
- que el puerto esté publicado por Docker;
- que el Security Group de EC2 permita el puerto;

- que se esté utilizando la IP pública actual de la instancia.

17. Consideraciones de seguridad

No deben almacenarse en el repositorio:

- credenciales de AWS;
- claves SSH;
- tokens;
- archivos `.env` con secretos;
- credenciales temporales de AWS Academy.

Las credenciales temporales deben cargarse únicamente mediante variables de entorno durante la sesión correspondiente.

18. Validación realizada

La instalación fue validada desplegando el stack completo en una instancia AWS EC2.

Se comprobó:

- frontend accesible públicamente;
- backend FastAPI saludable;
- inferencia utilizando el modelo empaquetado;
- integración tablero → API → modelo;
- MLflow accesible y persistente;
- versionado de datos con DVC;
- recuperación de datos desde S3 mediante `dvc pull`.